Lab1-Assignment

Copyright: Vrije Universiteit Amsterdam, Faculty of Humanities, CLTL

This notebook describes the assignment for Lab 1 of the text mining course.

Points: each exercise is prefixed with the number of points you can obtain for the exercise.

We assume you have worked through the following notebooks:

- **Lab1.1-introduction**
- **Lab1.2-introduction-to-NLTK**
- **Lab1.3-introduction-to-spaCy**

In this assignment, you will process an English text (**Lab1-apple-samsung-example.txt**) with both NLTK and spaCy and discuss the similarities and differences.

Credits

The notebooks in this block have been originally created by Marten Postma. Adaptations were made by Filip Ilievski.

Tip: how to read a file from disk

Let's open the file **Lab1-apple-samsung-example.txt** from disk.

In [169…

```
from pathlib import Path
```

In [170…

```
cur_dir = Path().resolve() # this should provide you with the folder in which this
path_to_file = Path.joinpath(cur_dir, 'Lab1-apple-samsung-example.txt')
print(path_to_file)
print('does path exist? ->', Path.exists(path_to_file))
```

```
C:\Users\dexter\Documents\VU_UvA\TMAI\ba-text-mining\lab_sessions\lab1\Lab1-apple-sa
msung-example.txt
does path exist? -> True
```

If the output from the code cell above states that **does path exist? -> False**, please check that the file **Lab1-apple-samsung-example.txt** is in the same directory as this notebook.

In [171…

```
with open(path_to_file) as infile:
    text = infile.read()

print('number of characters', len(text))
```

```
number of characters 1139
```

[total points: 4] Exercise 1: NLTK

In this exercise, we use NLTK to apply **Part-of-speech (POS) tagging**, **Named Entity Recognition (NER)**, and **Constituency parsing**. The following code snippet already performs sentence splitting and tokenization.

In [172…

```
import nltk
from nltk.tokenize import sent_tokenize
from nltk import word_tokenize
```

In [173…

```
sentences_nltk = sent_tokenize(text)
```

In [174…

```
tokens_per_sentence = []
for sentence_nltk in sentences_nltk:
    sent_tokens = word_tokenize(sentence_nltk)
    tokens_per_sentence.append(sent_tokens)
print(tokens_per_sentence)
```

```
[['https', ':', '//www.telegraph.co.uk/technology/apple/9702716/Apple-Samsung-lawsui
t-six-more-products-under-scrutiny.html', 'Documents', 'filed', 'to', 'the', 'San',
'Jose', 'federal', 'court', 'in', 'California', 'on', 'November', '23', 'list', 'si
x', 'Samsung', 'products', 'running', 'the', '``', 'Jelly', 'Bean', "''", 'and', '`
`', 'Ice', 'Cream', 'Sandwich', "''", 'operating', 'systems', ',', 'which', 'Apple',
'claims', 'infringe', 'its', 'patents', '.'], ['The', 'six', 'phones', 'and', 'table
ts', 'affected', 'are', 'the', 'Galaxy', 'S', 'III', ',', 'running', 'the', 'new',
'Jelly', 'Bean', 'system', ',', 'the', 'Galaxy', 'Tab', '8.9', 'Wifi', 'tablet',
',', 'the', 'Galaxy', 'Tab', '2', '10.1', ',', 'Galaxy', 'Rugby', 'Pro', 'and', 'Gal
axy', 'S', 'III', 'mini', '.'], ['Apple', 'stated', 'it', 'had', '“', 'acted', 'quic
kly', 'and', 'diligently', "''", 'in', 'order', 'to', '``', 'determine', 'that', 'th
ese', 'newly', 'released', 'products', 'do', 'infringe', 'many', 'of', 'the', 'sam
e', 'claims', 'already', 'asserted', 'by', 'Apple', '.', "''"], ['In', 'August',
',', 'Samsung', 'lost', 'a', 'US', 'patent', 'case', 'to', 'Apple', 'and', 'was', 'o
rdered', 'to', 'pay', 'its', 'rival', '$', '1.05bn', '(', '£0.66bn', ')', 'in', 'dam
ages', 'for', 'copying', 'features', 'of', 'the', 'iPad', 'and', 'iPhone', 'in', 'it
s', 'Galaxy', 'range', 'of', 'devices', '.'], ['Samsung', ',', 'which', 'is', 'the',
'world', "'s", 'top', 'mobile', 'phone', 'maker', ',', 'is', 'appealing', 'the', 'ru
ling', '.'], ['A', 'similar', 'case', 'in', 'the', 'UK', 'found', 'in', 'Samsung',
"'s", 'favour', 'and', 'ordered', 'Apple', 'to', 'publish', 'an', 'apology', 'makin
g', 'clear', 'that', 'the', 'South', 'Korean', 'firm', 'had', 'not', 'copied', 'it
s', 'iPad', 'when', 'designing', 'its', 'own', 'devices', '.']]
```

We will use lists to keep track of the output of the NLP tasks. We can hence inspect the output for each task using the index of the sentence.

In [175…

```
sent_id = 1
print('SENTENCE', sentences_nltk[sent_id])
print('TOKENS', tokens_per_sentence[sent_id])
```

```
SENTENCE The six phones and tablets affected are the Galaxy S III, running the new J
elly Bean system, the Galaxy Tab 8.9 Wifi tablet, the Galaxy Tab 2 10.1, Galaxy Rugb
y Pro and Galaxy S III mini.
TOKENS ['The', 'six', 'phones', 'and', 'tablets', 'affected', 'are', 'the', 'Galax
y', 'S', 'III', ',', 'running', 'the', 'new', 'Jelly', 'Bean', 'system', ',', 'the',
'Galaxy', 'Tab', '8.9', 'Wifi', 'tablet', ',', 'the', 'Galaxy', 'Tab', '2', '10.1',
',', 'Galaxy', 'Rugby', 'Pro', 'and', 'Galaxy', 'S', 'III', 'mini', '.']
```

[point: 1] Exercise 1a: Part-of-speech (POS) tagging

Use `nltk.pos_tag` to perform part-of-speech tagging on each sentence.

Use `print` to **show** the output in the notebook (and hence also in the exported PDF!).

In [176…

```
pos_tags_per_sentence = []
for tokens in tokens_per_sentence:
    pos_tags = nltk.pos_tag(tokens)
    print(pos_tags)
    pos_tags_per_sentence.append(pos_tags)
print()
print(pos_tags_per_sentence[1])
```

```
[('https', 'NN'), (':', ':'), ('//www.telegraph.co.uk/technology/apple/9702716/Apple
-Samsung-lawsuit-six-more-products-under-scrutiny.html', 'JJ'), ('Documents', 'NN
S'), ('filed', 'VBN'), ('to', 'TO'), ('the', 'DT'), ('San', 'NNP'), ('Jose', 'NNP'),
('federal', 'JJ'), ('court', 'NN'), ('in', 'IN'), ('California', 'NNP'), ('on', 'I
N'), ('November', 'NNP'), ('23', 'CD'), ('list', 'NN'), ('six', 'CD'), ('Samsung',
'NNP'), ('products', 'NNS'), ('running', 'VBG'), ('the', 'DT'), ('``', '``'), ('Jell
y', 'RB'), ('Bean', 'NNP'), ("''", "''"), ('and', 'CC'), ('``', '``'), ('Ice', 'NN
P'), ('Cream', 'NNP'), ('Sandwich', 'NNP'), ("''", "''"), ('operating', 'VBG'), ('sy
stems', 'NNS'), (',', ','), ('which', 'WDT'), ('Apple', 'NNP'), ('claims', 'VBZ'),
('infringe', 'VB'), ('its', 'PRP$'), ('patents', 'NNS'), ('.', '.')]
[('The', 'DT'), ('six', 'CD'), ('phones', 'NNS'), ('and', 'CC'), ('tablets', 'NNS'),
('affected', 'VBN'), ('are', 'VBP'), ('the', 'DT'), ('Galaxy', 'NNP'), ('S', 'NNP'),
('III', 'NNP'), (',', ','), ('running', 'VBG'), ('the', 'DT'), ('new', 'JJ'), ('Jell
y', 'NNP'), ('Bean', 'NNP'), ('system', 'NN'), (',', ','), ('the', 'DT'), ('Galaxy',
'NNP'), ('Tab', 'NNP'), ('8.9', 'CD'), ('Wifi', 'NNP'), ('tablet', 'NN'), (',',
','), ('the', 'DT'), ('Galaxy', 'NNP'), ('Tab', 'NNP'), ('2', 'CD'), ('10.1', 'CD'),
(',', ','), ('Galaxy', 'NNP'), ('Rugby', 'NNP'), ('Pro', 'NNP'), ('and', 'CC'), ('Ga
laxy', 'NNP'), ('S', 'NNP'), ('III', 'NNP'), ('mini', 'NN'), ('.', '.')]
[('Apple', 'NNP'), ('stated', 'VBD'), ('it', 'PRP'), ('had', 'VBD'), ('“', 'NNP'),
('acted', 'VBD'), ('quickly', 'RB'), ('and', 'CC'), ('diligently', 'RB'), ("''",
"''"), ('in', 'IN'), ('order', 'NN'), ('to', 'TO'), ('``', '``'), ('determine', 'V
B'), ('that', 'IN'), ('these', 'DT'), ('newly', 'RB'), ('released', 'VBN'), ('produc
ts', 'NNS'), ('do', 'VBP'), ('infringe', 'VB'), ('many', 'JJ'), ('of', 'IN'), ('th
e', 'DT'), ('same', 'JJ'), ('claims', 'NNS'), ('already', 'RB'), ('asserted', 'VB
N'), ('by', 'IN'), ('Apple', 'NNP'), ('.', '.'), ("''", "''")]
[('In', 'IN'), ('August', 'NNP'), (',', ','), ('Samsung', 'NNP'), ('lost', 'VBD'),
('a', 'DT'), ('US', 'NNP'), ('patent', 'NN'), ('case', 'NN'), ('to', 'TO'), ('Appl
e', 'NNP'), ('and', 'CC'), ('was', 'VBD'), ('ordered', 'VBN'), ('to', 'TO'), ('pay',
'VB'), ('its', 'PRP$'), ('rival', 'JJ'), ('$', '$'), ('1.05bn', 'CD'), ('(', '('),
('£0.66bn', 'NN'), (')', ')'), ('in', 'IN'), ('damages', 'NNS'), ('for', 'IN'), ('co
pying', 'VBG'), ('features', 'NNS'), ('of', 'IN'), ('the', 'DT'), ('iPad', 'NN'),
('and', 'CC'), ('iPhone', 'NN'), ('in', 'IN'), ('its', 'PRP$'), ('Galaxy', 'NNP'),
('range', 'NN'), ('of', 'IN'), ('devices', 'NNS'), ('.', '.')]
[('Samsung', 'NNP'), (',', ','), ('which', 'WDT'), ('is', 'VBZ'), ('the', 'DT'), ('w
orld', 'NN'), ("'s", 'POS'), ('top', 'JJ'), ('mobile', 'NN'), ('phone', 'NN'), ('mak
er', 'NN'), (',', ','), ('is', 'VBZ'), ('appealing', 'VBG'), ('the', 'DT'), ('rulin
g', 'NN'), ('.', '.')]
[('A', 'DT'), ('similar', 'JJ'), ('case', 'NN'), ('in', 'IN'), ('the', 'DT'), ('UK',
'NNP'), ('found', 'VBD'), ('in', 'IN'), ('Samsung', 'NNP'), ("'s", 'POS'), ('favou
r', 'NN'), ('and', 'CC'), ('ordered', 'VBD'), ('Apple', 'NNP'), ('to', 'TO'), ('publ
ish', 'VB'), ('an', 'DT'), ('apology', 'NN'), ('making', 'VBG'), ('clear', 'JJ'),
('that', 'IN'), ('the', 'DT'), ('South', 'JJ'), ('Korean', 'JJ'), ('firm', 'NN'),
('had', 'VBD'), ('not', 'RB'), ('copied', 'VBN'), ('its', 'PRP$'), ('iPad', 'NN'),
('when', 'WRB'), ('designing', 'VBG'), ('its', 'PRP$'), ('own', 'JJ'), ('devices',
'NNS'), ('.', '.')]
```


```
[('The', 'DT'), ('six', 'CD'), ('phones', 'NNS'), ('and', 'CC'), ('tablets', 'NNS'),
('affected', 'VBN'), ('are', 'VBP'), ('the', 'DT'), ('Galaxy', 'NNP'), ('S', 'NNP'),
('III', 'NNP'), (',', ','), ('running', 'VBG'), ('the', 'DT'), ('new', 'JJ'), ('Jell
y', 'NNP'), ('Bean', 'NNP'), ('system', 'NN'), (',', ','), ('the', 'DT'), ('Galaxy',
'NNP'), ('Tab', 'NNP'), ('8.9', 'CD'), ('Wifi', 'NNP'), ('tablet', 'NN'), (',',
','), ('the', 'DT'), ('Galaxy', 'NNP'), ('Tab', 'NNP'), ('2', 'CD'), ('10.1', 'CD'),
(',', ','), ('Galaxy', 'NNP'), ('Rugby', 'NNP'), ('Pro', 'NNP'), ('and', 'CC'), ('Ga
laxy', 'NNP'), ('S', 'NNP'), ('III', 'NNP'), ('mini', 'NN'), ('.', '.')]
```

[point: 1] Exercise 1b: Named Entity Recognition (NER)

Use `nltk.chunk.ne_chunk` to perform Named Entity Recognition (NER) on each sentence.

Use `print` to **show** the output in the notebook (and hence also in the exported PDF!).

In [177…

```
ner_tags_per_sentence = []
for pos_tags in pos_tags_per_sentence:
    ner_chunk = nltk.chunk.ne_chunk(pos_tags)
    ner_tags_per_sentence.append(ner_chunk)
print(ner_tags_per_sentence)
```

```
[Tree('S', [('https', 'NN'), (':', ':'), ('//www.telegraph.co.uk/technology/apple/97
02716/Apple-Samsung-lawsuit-six-more-products-under-scrutiny.html', 'JJ'), ('Documen
ts', 'NNS'), ('filed', 'VBN'), ('to', 'TO'), ('the', 'DT'), Tree('ORGANIZATION',
[('San', 'NNP'), ('Jose', 'NNP')]), ('federal', 'JJ'), ('court', 'NN'), ('in', 'I
N'), Tree('GPE', [('California', 'NNP')]), ('on', 'IN'), ('November', 'NNP'), ('23',
'CD'), ('list', 'NN'), ('six', 'CD'), Tree('ORGANIZATION', [('Samsung', 'NNP')]),
('products', 'NNS'), ('running', 'VBG'), ('the', 'DT'), ('``', '``'), ('Jelly', 'R
B'), Tree('GPE', [('Bean', 'NNP')]), ("''", "''"), ('and', 'CC'), ('``', '``'), ('Ic
e', 'NNP'), ('Cream', 'NNP'), ('Sandwich', 'NNP'), ("''", "''"), ('operating', 'VB
G'), ('systems', 'NNS'), (',', ','), ('which', 'WDT'), Tree('PERSON', [('Apple', 'NN
P')]), ('claims', 'VBZ'), ('infringe', 'VB'), ('its', 'PRP$'), ('patents', 'NNS'),
('.', '.')]), Tree('S', [('The', 'DT'), ('six', 'CD'), ('phones', 'NNS'), ('and', 'C
C'), ('tablets', 'NNS'), ('affected', 'VBN'), ('are', 'VBP'), ('the', 'DT'), Tree('O
RGANIZATION', [('Galaxy', 'NNP')]), ('S', 'NNP'), ('III', 'NNP'), (',', ','), ('runn
ing', 'VBG'), ('the', 'DT'), ('new', 'JJ'), Tree('PERSON', [('Jelly', 'NNP'), ('Bea
n', 'NNP')]), ('system', 'NN'), (',', ','), ('the', 'DT'), Tree('ORGANIZATION', [('G
alaxy', 'NNP')]), ('Tab', 'NNP'), ('8.9', 'CD'), ('Wifi', 'NNP'), ('tablet', 'NN'),
(',', ','), ('the', 'DT'), Tree('ORGANIZATION', [('Galaxy', 'NNP')]), ('Tab', 'NN
P'), ('2', 'CD'), ('10.1', 'CD'), (',', ','), Tree('PERSON', [('Galaxy', 'NNP'), ('R
ugby', 'NNP'), ('Pro', 'NNP')]), ('and', 'CC'), Tree('PERSON', [('Galaxy', 'NNP'),
('S', 'NNP')]), ('III', 'NNP'), ('mini', 'NN'), ('.', '.')]), Tree('S', [Tree('PERSO
N', [('Apple', 'NNP')]), ('stated', 'VBD'), ('it', 'PRP'), ('had', 'VBD'), ('“', 'NN
P'), ('acted', 'VBD'), ('quickly', 'RB'), ('and', 'CC'), ('diligently', 'RB'),
("''", "''"), ('in', 'IN'), ('order', 'NN'), ('to', 'TO'), ('``', '``'), ('determin
e', 'VB'), ('that', 'IN'), ('these', 'DT'), ('newly', 'RB'), ('released', 'VBN'),
('products', 'NNS'), ('do', 'VBP'), ('infringe', 'VB'), ('many', 'JJ'), ('of', 'I
N'), ('the', 'DT'), ('same', 'JJ'), ('claims', 'NNS'), ('already', 'RB'), ('asserte
d', 'VBN'), ('by', 'IN'), Tree('PERSON', [('Apple', 'NNP')]), ('.', '.'), ("''",
"'')]), Tree('S', [('In', 'IN'), Tree('GPE', [('August', 'NNP')]), (',', ','), Tree
('PERSON', [('Samsung', 'NNP')]), ('lost', 'VBD'), ('a', 'DT'), Tree('GSP', [('US',
'NNP')]), ('patent', 'NN'), ('case', 'NN'), ('to', 'TO'), Tree('GPE', [('Apple', 'NN
P')]), ('and', 'CC'), ('was', 'VBD'), ('ordered', 'VBN'), ('to', 'TO'), ('pay', 'V
B'), ('its', 'PRP$'), ('rival', 'JJ'), ('$', '$'), ('1.05bn', 'CD'), ('(', '('),
('£0.66bn', 'NN'), (')', ')'), ('in', 'IN'), ('damages', 'NNS'), ('for', 'IN'), ('co
pying', 'VBG'), ('features', 'NNS'), ('of', 'IN'), ('the', 'DT'), Tree('ORGANIZATIO
N', [('iPad', 'NN')]), ('and', 'CC'), Tree('ORGANIZATION', [('iPhone', 'NN')]), ('i
n', 'IN'), ('its', 'PRP$'), Tree('GPE', [('Galaxy', 'NNP')]), ('range', 'NN'), ('o
f', 'IN'), ('devices', 'NNS'), ('.', '.')]), Tree('S', [Tree('GPE', [('Samsung', 'NN
P')]), (',', ','), ('which', 'WDT'), ('is', 'VBZ'), ('the', 'DT'), ('world', 'NN'),
("'s", 'POS'), ('top', 'JJ'), ('mobile', 'NN'), ('phone', 'NN'), ('maker', 'NN'),
(',', ','), ('is', 'VBZ'), ('appealing', 'VBG'), ('the', 'DT'), ('ruling', 'NN'),
('.', '.')]), Tree('S', [('A', 'DT'), ('similar', 'JJ'), ('case', 'NN'), ('in', 'I
N'), ('the', 'DT'), Tree('ORGANIZATION', [('UK', 'NNP')]), ('found', 'VBD'), ('in',
'IN'), Tree('GPE', [('Samsung', 'NNP')]), ("'s", 'POS'), ('favour', 'NN'), ('and',
'CC'), ('ordered', 'VBD'), Tree('PERSON', [('Apple', 'NNP')]), ('to', 'TO'), ('publi
sh', 'VB'), ('an', 'DT'), ('apology', 'NN'), ('making', 'VBG'), ('clear', 'JJ'), ('t
hat', 'IN'), ('the', 'DT'), Tree('LOCATION', [('South', 'JJ'), ('Korean', 'JJ')]),
('firm', 'NN'), ('had', 'VBD'), ('not', 'RB'), ('copied', 'VBN'), ('its', 'PRP$'),
('iPad', 'NN'), ('when', 'WRB'), ('designing', 'VBG'), ('its', 'PRP$'), ('own', 'J
J'), ('devices', 'NNS'), ('.', '.')])]
```

[points: 2] Exercise 1c: Constituency parsing

Use the `nltk.RegexpParser` to perform constituency parsing on each sentence.

Use `print` to **show** the output in the notebook (and hence also in the exported PDF!).

In [178…

```
constituent_parser = nltk.RegexpParser('''
NP: {<DT>? <JJ>* <NN>*} # NP
P: {<IN>}           # Preposition
V: {<V.*>}          # Verb
PP: {<P> <NP>}      # PP -> P NP
VP: {<V> <NP|PP>*}  # VP -> V (NP|PP)*''')
```

In [179…

```
constituency_output_per_sentence = []
for pos_tags in pos_tags_per_sentence:
    tokens_constituency_parsed = constituent_parser.parse(pos_tags)
    constituency_output_per_sentence.append(tokens_constituency_parsed)
```

In [180…

```
print(constituency_output_per_sentence)
```

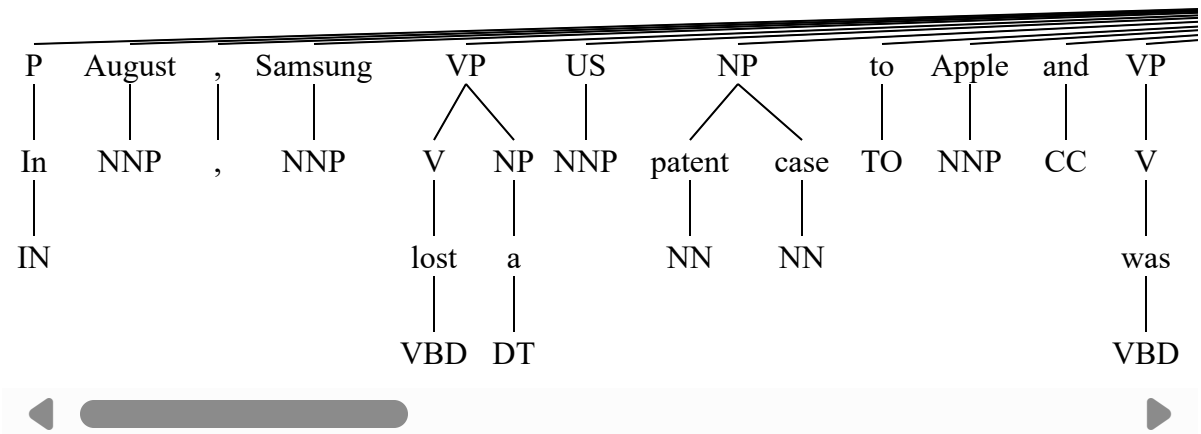
[Tree('S', [Tree('NP', [('https', 'NN')]), (':', ':'), Tree('NP', [('//www.telegrap
h.co.uk/technology/apple/9702716/Apple-Samsung-lawsuit-six-more-products-under-scrut
iny.html', 'JJ')]), ('Documents', 'NNS'), Tree('VP', [Tree('V', [('filed', 'VB
N')])]), ('to', 'TO'), Tree('NP', [('the', 'DT')]), ('San', 'NNP'), ('Jose', 'NNP'),
Tree('NP', [('federal', 'JJ'), ('court', 'NN')]), Tree('P', [('in', 'IN')]), ('Calif
ornia', 'NNP'), Tree('P', [('on', 'IN')]), ('November', 'NNP'), ('23', 'CD'), Tree
('NP', [('list', 'NN')]), ('six', 'CD'), ('Samsung', 'NNP'), ('products', 'NNS'), Tr
ee('VP', [Tree('V', [('running', 'VBG')]), Tree('NP', [('the', 'DT')])]), ('``', '`
`'), ('Jelly', 'RB'), ('Bean', 'NNP'), ("''", "''"), ('and', 'CC'), ('``', '``'),
('Ice', 'NNP'), ('Cream', 'NNP'), ('Sandwich', 'NNP'), ("''", "''"), Tree('VP', [Tre
e('V', [('operating', 'VBG')])]), ('systems', 'NNS'), (',', ','), ('which', 'WDT'),
('Apple', 'NNP'), Tree('VP', [Tree('V', [('claims', 'VBZ')])]), Tree('VP', [Tree
('V', [('infringe', 'VB')])]), ('its', 'PRP$'), ('patents', 'NNS'), ('.', '.')]), Tr
ee('S', [Tree('NP', [('The', 'DT')]), ('six', 'CD'), ('phones', 'NNS'), ('and', 'C
C'), ('tablets', 'NNS'), Tree('VP', [Tree('V', [('affected', 'VBN')])]), Tree('VP',
[Tree('V', [('are', 'VBP')]), Tree('NP', [('the', 'DT')])]), ('Galaxy', 'NNP'),
('S', 'NNP'), ('III', 'NNP'), (',', ','), Tree('VP', [Tree('V', [('running', 'VB
G')]), Tree('NP', [('the', 'DT'), ('new', 'JJ')])]), ('Jelly', 'NNP'), ('Bean', 'NN
P'), Tree('NP', [('system', 'NN')]), (',', ','), Tree('NP', [('the', 'DT')]), ('Gala
xy', 'NNP'), ('Tab', 'NNP'), ('8.9', 'CD'), ('Wifi', 'NNP'), Tree('NP', [('tablet',
'NN')]), (',', ','), Tree('NP', [('the', 'DT')]), ('Galaxy', 'NNP'), ('Tab', 'NNP'),
('2', 'CD'), ('10.1', 'CD'), (',', ','), ('Galaxy', 'NNP'), ('Rugby', 'NNP'), ('Pr
o', 'NNP'), ('and', 'CC'), ('Galaxy', 'NNP'), ('S', 'NNP'), ('III', 'NNP'), Tree('N
P', [('mini', 'NN')]), ('.', '.')]), Tree('S', [('Apple', 'NNP'), Tree('VP', [Tree
('V', [('stated', 'VBD')])]), ('it', 'PRP'), Tree('VP', [Tree('V', [('had', 'VB
D')])]), ('“', 'NNP'), Tree('VP', [Tree('V', [('acted', 'VBD')])]), ('quickly', 'R
B'), ('and', 'CC'), ('diligently', 'RB'), ("''", "''"), Tree('PP', [Tree('P', [('i
n', 'IN')]), Tree('NP', [('order', 'NN')])]), ('to', 'TO'), ('``', '``'), Tree('VP',
[Tree('V', [('determine', 'VB')]), Tree('PP', [Tree('P', [('that', 'IN')]), Tree('N
P', [('these', 'DT')])])]), ('newly', 'RB'), Tree('VP', [Tree('V', [('released', 'VB
N')])]), ('products', 'NNS'), Tree('VP', [Tree('V', [('do', 'VBP')])]), Tree('VP',
[Tree('V', [('infringe', 'VB')]), Tree('NP', [('many', 'JJ')]), Tree('PP', [Tree
('P', [('of', 'IN')]), Tree('NP', [('the', 'DT'), ('same', 'JJ')])])]), ('claims',
'NNS'), ('already', 'RB'), Tree('VP', [Tree('V', [('asserted', 'VBN')])]), Tree('P',
[('by', 'IN')]), ('Apple', 'NNP'), ('.', '.'), ("''", "''")]), Tree('S', [Tree('P',
[('In', 'IN')]), ('August', 'NNP'), (',', ','), ('Samsung', 'NNP'), Tree('VP', [Tree
('V', [('lost', 'VBD')]), Tree('NP', [('a', 'DT')])]), ('US', 'NNP'), Tree('NP',
[('patent', 'NN'), ('case', 'NN')]), ('to', 'TO'), ('Apple', 'NNP'), ('and', 'CC'),
Tree('VP', [Tree('V', [('was', 'VBD')])]), Tree('VP', [Tree('V', [('ordered', 'VB
N')])]), ('to', 'TO'), Tree('VP', [Tree('V', [('pay', 'VB')])]), ('its', 'PRP$'), Tr
ee('NP', [('rival', 'JJ')]), ('$', '$'), ('1.05bn', 'CD'), ('(', '('), Tree('NP',
[('£0.66bn', 'NN')]), (')', ')'), Tree('P', [('in', 'IN')]), ('damages', 'NNS'), Tre
e('P', [('for', 'IN')]), Tree('VP', [Tree('V', [('copying', 'VBG')])]), ('features',
'NNS'), Tree('PP', [Tree('P', [('of', 'IN')]), Tree('NP', [('the', 'DT'), ('iPad',
'NN')])]), ('and', 'CC'), Tree('NP', [('iPhone', 'NN')]), Tree('P', [('in', 'IN')]),
('its', 'PRP$'), ('Galaxy', 'NNP'), Tree('NP', [('range', 'NN')]), Tree('P', [('of',
'IN')]), ('devices', 'NNS'), ('.', '.')]), Tree('S', [('Samsung', 'NNP'), (',',
','), ('which', 'WDT'), Tree('VP', [Tree('V', [('is', 'VBZ')]), Tree('NP', [('the',
'DT'), ('world', 'NN')])]), ("'s", 'POS'), Tree('NP', [('top', 'JJ'), ('mobile', 'N
N'), ('phone', 'NN'), ('maker', 'NN')]), (',', ','), Tree('VP', [Tree('V', [('is',
'VBZ')])]), Tree('VP', [Tree('V', [('appealing', 'VBG')]), Tree('NP', [('the', 'D
T'), ('ruling', 'NN')])]), ('.', '.')]), Tree('S', [Tree('NP', [('A', 'DT'), ('simil
ar', 'JJ'), ('case', 'NN')]), Tree('PP', [Tree('P', [('in', 'IN')]), Tree('NP', [('t
he', 'DT')])]), ('UK', 'NNP'), Tree('VP', [Tree('V', [('found', 'VBD')])]), Tree
('P', [('in', 'IN')]), ('Samsung', 'NNP'), ("'s", 'POS'), Tree('NP', [('favour', 'N
N')]), ('and', 'CC'), Tree('VP', [Tree('V', [('ordered', 'VBD')])]), ('Apple', 'NN

```
P'), ('to', 'TO'), Tree('VP', [Tree('V', [('publish', 'VB')]), Tree('NP', [('an', 'D
T'), ('apology', 'NN')])]), Tree('VP', [Tree('V', [('making', 'VBG')]), Tree('NP',
[('clear', 'JJ')]), Tree('PP', [Tree('P', [('that', 'IN')]), Tree('NP', [('the', 'D
T'), ('South', 'JJ'), ('Korean', 'JJ'), ('firm', 'NN')])])]), Tree('VP', [Tree('V',
[('had', 'VBD')])]), ('not', 'RB'), Tree('VP', [Tree('V', [('copied', 'VBN')])]),
('its', 'PRP$'), Tree('NP', [('iPad', 'NN')]), ('when', 'WRB'), Tree('VP', [Tree
('V', [('designing', 'VBG')])]), ('its', 'PRP$'), Tree('NP', [('own', 'JJ')]), ('dev
ices', 'NNS'), ('.', '.')])]
```

In [196…

```
constituency_output_per_sentence[3]
```

Out[196…



Augment the RegexpParser so that it also detects Named Entity Phrases (NEP), e.g., that it detects *Galaxy S III* and *Ice Cream Sandwich*

In [181…

```
constituent_parser_v2 = nltk.RegexpParser('''
NP: {<DT>? <JJ>* <NN>*} # NP
P: {<IN>}           # Preposition
V: {<V.*>}          # Verb
PP: {<P> <NP>}      # PP -> P NP
VP: {<V> <NP|PP>*}  # VP -> V (NP|PP)*
NEP: {<NNP>+}             # Multiple NNP''')
```

In [182…

```
constituency_v2_output_per_sentence = []
for pos_tags in pos_tags_per_sentence:
    tokens_constituency_parsed_v2 = constituent_parser_v2.parse(pos_tags)
    constituency_v2_output_per_sentence.append(tokens_constituency_parsed_v2)
```

In [183…

```
print(constituency_v2_output_per_sentence)
```

[Tree('S', [Tree('NP', [('https', 'NN')]), (':', ':'), Tree('NP', [('//www.telegrap
h.co.uk/technology/apple/9702716/Apple-Samsung-lawsuit-six-more-products-under-scrut
iny.html', 'JJ')]), ('Documents', 'NNS'), Tree('VP', [Tree('V', [('filed', 'VB
N')])]), ('to', 'TO'), Tree('NP', [('the', 'DT')]), Tree('NEP', [('San', 'NNP'), ('J
ose', 'NNP')]), Tree('NP', [('federal', 'JJ'), ('court', 'NN')]), Tree('P', [('in',
'IN')]), Tree('NEP', [('California', 'NNP')]), Tree('P', [('on', 'IN')]), Tree('NE
P', [('November', 'NNP')]), ('23', 'CD'), Tree('NP', [('list', 'NN')]), ('six', 'C
D'), Tree('NEP', [('Samsung', 'NNP')]), ('products', 'NNS'), Tree('VP', [Tree('V',
[('running', 'VBG')]), Tree('NP', [('the', 'DT')])]), ('``', '``'), ('Jelly', 'RB'),
Tree('NEP', [('Bean', 'NNP')]), ("''", "''"), ('and', 'CC'), ('``', '``'), Tree('NE
P', [('Ice', 'NNP'), ('Cream', 'NNP'), ('Sandwich', 'NNP')]), ("''", "''"), Tree('V
P', [Tree('V', [('operating', 'VBG')])]), ('systems', 'NNS'), (',', ','), ('which',
'WDT'), Tree('NEP', [('Apple', 'NNP')]), Tree('VP', [Tree('V', [('claims', 'VB
Z')])]), Tree('VP', [Tree('V', [('infringe', 'VB')])]), ('its', 'PRP$'), ('patents',
'NNS'), ('.', '.')]), Tree('S', [Tree('NP', [('The', 'DT')]), ('six', 'CD'), ('phone
s', 'NNS'), ('and', 'CC'), ('tablets', 'NNS'), Tree('VP', [Tree('V', [('affected',
'VBN')])]), Tree('VP', [Tree('V', [('are', 'VBP')]), Tree('NP', [('the', 'DT')])]),
Tree('NEP', [('Galaxy', 'NNP'), ('S', 'NNP'), ('III', 'NNP')]), (',', ','), Tree('V
P', [Tree('V', [('running', 'VBG')]), Tree('NP', [('the', 'DT'), ('new', 'JJ')])]),
Tree('NEP', [('Jelly', 'NNP'), ('Bean', 'NNP')]), Tree('NP', [('system', 'NN')]),
(',', ','), Tree('NP', [('the', 'DT')]), Tree('NEP', [('Galaxy', 'NNP'), ('Tab', 'NN
P')]), ('8.9', 'CD'), Tree('NEP', [('Wifi', 'NNP')]), Tree('NP', [('tablet', 'N
N')]), (',', ','), Tree('NP', [('the', 'DT')]), Tree('NEP', [('Galaxy', 'NNP'), ('Ta
b', 'NNP')]), ('2', 'CD'), ('10.1', 'CD'), (',', ','), Tree('NEP', [('Galaxy', 'NN
P'), ('Rugby', 'NNP'), ('Pro', 'NNP')]), ('and', 'CC'), Tree('NEP', [('Galaxy', 'NN
P'), ('S', 'NNP'), ('III', 'NNP')]), Tree('NP', [('mini', 'NN')]), ('.', '.')]), Tre
e('S', [Tree('NEP', [('Apple', 'NNP')]), Tree('VP', [Tree('V', [('stated', 'VB
D')])]), ('it', 'PRP'), Tree('VP', [Tree('V', [('had', 'VBD')])]), Tree('NEP',
[('“', 'NNP')]), Tree('VP', [Tree('V', [('acted', 'VBD')])]), ('quickly', 'RB'), ('a
nd', 'CC'), ('diligently', 'RB'), ("''", "''"), Tree('PP', [Tree('P', [('in', 'I
N')]), Tree('NP', [('order', 'NN')])]), ('to', 'TO'), ('``', '``'), Tree('VP', [Tree
('V', [('determine', 'VB')]), Tree('PP', [Tree('P', [('that', 'IN')]), Tree('NP',
[('these', 'DT')])])]), ('newly', 'RB'), Tree('VP', [Tree('V', [('released', 'VB
N')])]), ('products', 'NNS'), Tree('VP', [Tree('V', [('do', 'VBP')])]), Tree('VP',
[Tree('V', [('infringe', 'VB')]), Tree('NP', [('many', 'JJ')]), Tree('PP', [Tree
('P', [('of', 'IN')]), Tree('NP', [('the', 'DT'), ('same', 'JJ')])])]), ('claims',
'NNS'), ('already', 'RB'), Tree('VP', [Tree('V', [('asserted', 'VBN')])]), Tree('P',
[('by', 'IN')]), Tree('NEP', [('Apple', 'NNP')]), ('.', '.'), ("''", "''")]), Tree
('S', [Tree('P', [('In', 'IN')]), Tree('NEP', [('August', 'NNP')]), (',', ','), Tree
('NEP', [('Samsung', 'NNP')]), Tree('VP', [Tree('V', [('lost', 'VBD')]), Tree('NP',
[('a', 'DT')])]), Tree('NEP', [('US', 'NNP')]), Tree('NP', [('patent', 'NN'), ('cas
e', 'NN')]), ('to', 'TO'), Tree('NEP', [('Apple', 'NNP')]), ('and', 'CC'), Tree('V
P', [Tree('V', [('was', 'VBD')])]), Tree('VP', [Tree('V', [('ordered', 'VBN')])]),
('to', 'TO'), Tree('VP', [Tree('V', [('pay', 'VB')])]), ('its', 'PRP$'), Tree('NP',
[('rival', 'JJ')]), ('$', '$'), ('1.05bn', 'CD'), ('(', '('), Tree('NP', [('£0.66b
n', 'NN')]), (')', ')'), Tree('P', [('in', 'IN')]), ('damages', 'NNS'), Tree('P',
[('for', 'IN')]), Tree('VP', [Tree('V', [('copying', 'VBG')])]), ('features', 'NN
S'), Tree('PP', [Tree('P', [('of', 'IN')]), Tree('NP', [('the', 'DT'), ('iPad', 'N
N')])]), ('and', 'CC'), Tree('NP', [('iPhone', 'NN')]), Tree('P', [('in', 'IN')]),
('its', 'PRP$'), Tree('NEP', [('Galaxy', 'NNP')]), Tree('NP', [('range', 'NN')]), Tr
ee('P', [('of', 'IN')]), ('devices', 'NNS'), ('.', '.')]), Tree('S', [Tree('NEP',
[('Samsung', 'NNP')]), (',', ','), ('which', 'WDT'), Tree('VP', [Tree('V', [('is',
'VBZ')]), Tree('NP', [('the', 'DT'), ('world', 'NN')])]), ("'s", 'POS'), Tree('NP',
[('top', 'JJ'), ('mobile', 'NN'), ('phone', 'NN'), ('maker', 'NN')]), (',', ','), Tr
ee('VP', [Tree('V', [('is', 'VBZ')])]), Tree('VP', [Tree('V', [('appealing', 'VB
G')]), Tree('NP', [('the', 'DT'), ('ruling', 'NN')])]), ('.', '.')]), Tree('S', [Tre

```
e('NP', [('A', 'DT'), ('similar', 'JJ'), ('case', 'NN')]), Tree('PP', [Tree('P',
[('in', 'IN')]), Tree('NP', [('the', 'DT')])]), Tree('NEP', [('UK', 'NNP')]), Tree
('VP', [Tree('V', [('found', 'VBD')])]), Tree('P', [('in', 'IN')]), Tree('NEP', [('S
amsung', 'NNP')]), ("'s", 'POS'), Tree('NP', [('favour', 'NN')]), ('and', 'CC'), Tre
e('VP', [Tree('V', [('ordered', 'VBD')])]), Tree('NEP', [('Apple', 'NNP')]), ('to',
'TO'), Tree('VP', [Tree('V', [('publish', 'VB')]), Tree('NP', [('an', 'DT'), ('apolo
gy', 'NN')])]), Tree('VP', [Tree('V', [('making', 'VBG')]), Tree('NP', [('clear', 'J
J')]), Tree('PP', [Tree('P', [('that', 'IN')]), Tree('NP', [('the', 'DT'), ('South',
'JJ'), ('Korean', 'JJ'), ('firm', 'NN')])])]), Tree('VP', [Tree('V', [('had', 'VB
D')])]), ('not', 'RB'), Tree('VP', [Tree('V', [('copied', 'VBN')])]), ('its', 'PRP
$'), Tree('NP', [('iPad', 'NN')]), ('when', 'WRB'), Tree('VP', [Tree('V', [('designi
ng', 'VBG')])]), ('its', 'PRP$'), Tree('NP', [('own', 'JJ')]), ('devices', 'NNS'),
('.', '.')])]
```

[total points: 1] Exercise 2: spaCy

Use Spacy to process the same text as you analyzed with NLTK.

In [184…

```
import spacy
from spacy import displacy
nlp = spacy.load('en_core_web_sm')
```

In [185…

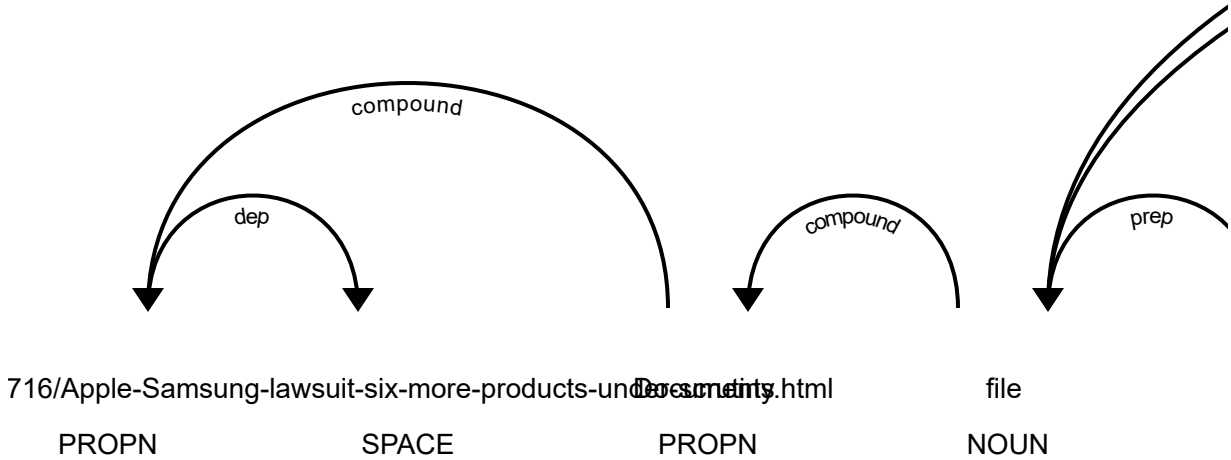
```
print(type(sentences_spacy[1]))
print(type(sentences_spacy_lemmatized[1]))
```

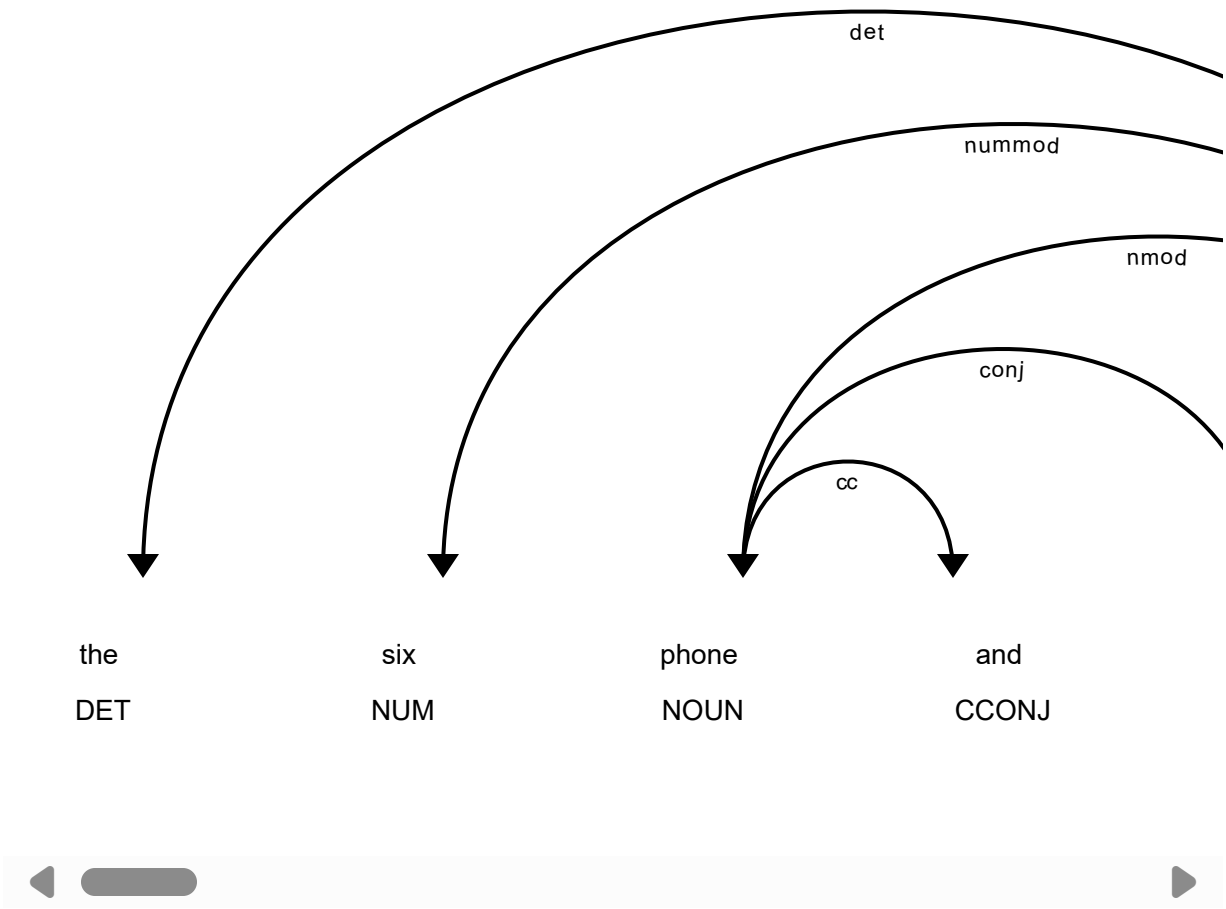
```
<class 'spacy.tokens.span.Span'>
<class 'spacy.tokens.doc.Doc'>
```

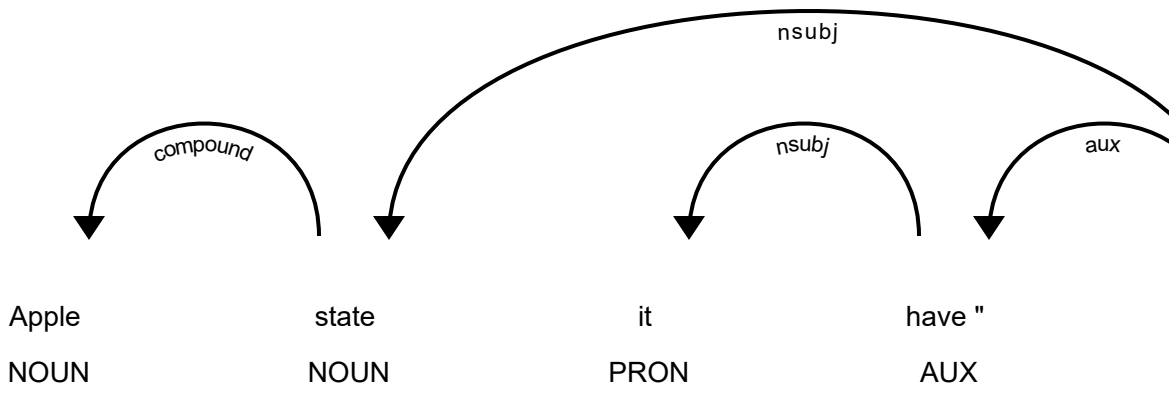
In [186…

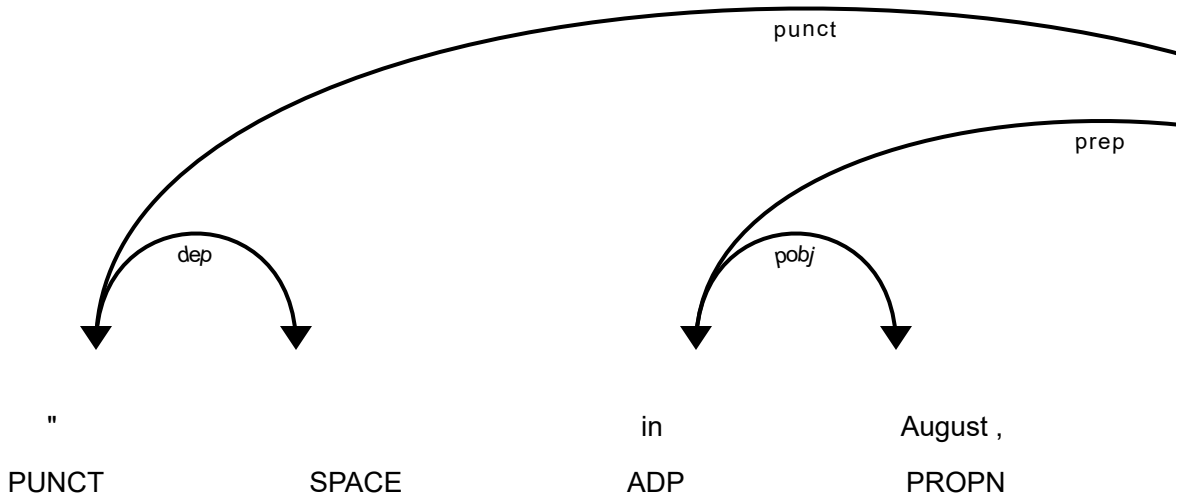
```
doc = nlp(text)
sentences_spacy = list(doc.sents)
print(sentences_spacy[1])
# lemmatization
sentences_spacy_lemmatized = []
for sentence in sentences_spacy:
    sent_lemmas = [token.lemma_ for token in sentence]
    sent_lemmas_lemmatized = ' '.join(sent_lemmas)
    doc_lemmatized = nlp(sent_lemmas_lemmatized)
    sentences_spacy_lemmatized.append(doc_lemmatized)
print(sentences_spacy_lemmatized[1])
# tagging
sentences_spacy_lemmatized_tagged = []
for sentence in sentences_spacy_lemmatized:
    sent_tags = [(token.text, token.pos_) for token in sentence]
    sentences_spacy_lemmatized_tagged.append(sent_tags)
print(sentences_spacy_lemmatized_tagged[1])
# dependency parsing
sentences_spacy_lemmatized_tagged_parsed = []
for sentence in sentences_spacy_lemmatized:
    sent_parsed = [(token.text, token.dep_) for token in sentence]
    displacy.render(sentence, style='dep', jupyter=True, options={'distance': 150})
    sentences_spacy_lemmatized_tagged_parsed.append(sent_parsed)
print(sentences_spacy_lemmatized_tagged_parsed[1])
```

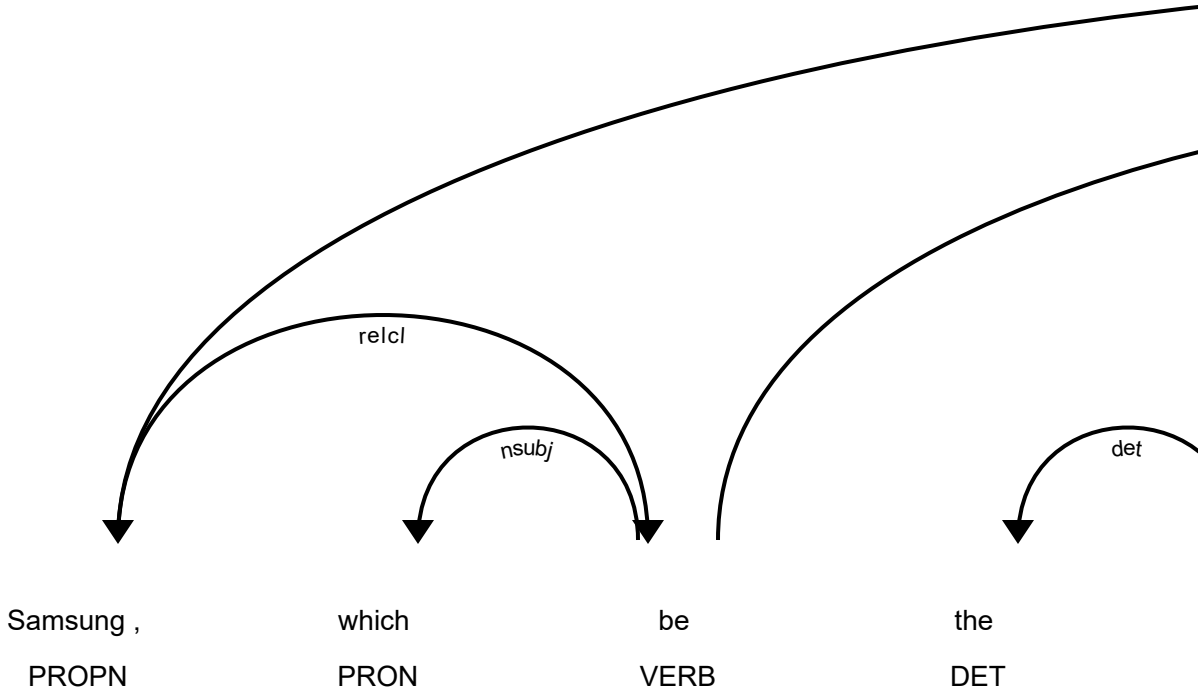
```
The six phones and tablets affected are the Galaxy S III, running the new Jelly Bean
system, the Galaxy Tab 8.9 Wifi tablet, the Galaxy Tab 2 10.1, Galaxy Rugby Pro and
Galaxy S III mini.

the six phone and tablet affect be the Galaxy S III , run the new Jelly Bean system
, the Galaxy Tab 8.9 Wifi tablet , the Galaxy Tab 2 10.1 , Galaxy Rugby Pro and Gala
xy S III mini .

[('the', 'DET'), ('six', 'NUM'), ('phone', 'NOUN'), ('and', 'CCONJ'), ('tablet', 'NO
UN'), ('affect', 'NOUN'), ('be', 'AUX'), ('the', 'DET'), ('Galaxy', 'PROPN'), ('S',
'PROPN'), ('III', 'PROPN'), (',', 'PUNCT'), ('run', 'VERB'), ('the', 'DET'), ('new',
'ADJ'), ('Jelly', 'PROPN'), ('Bean', 'PROPN'), ('system', 'NOUN'), (',', 'PUNCT'),
('the', 'DET'), ('Galaxy', 'PROPN'), ('Tab', 'PROPN'), ('8.9', 'NUM'), ('Wifi', 'PRO
PN'), ('tablet', 'NOUN'), (',', 'PUNCT'), ('the', 'DET'), ('Galaxy', 'PROPN'), ('Ta
b', 'PROPN'), ('2', 'NUM'), ('10.1', 'NUM'), (',', 'PUNCT'), ('Galaxy', 'PROPN'),
('Rugby', 'PROPN'), ('Pro', 'PROPN'), ('and', 'CCONJ'), ('Galaxy', 'PROPN'), ('S',
'PROPN'), ('III', 'PROPN'), ('mini', 'NOUN'), ('.', 'PUNCT'), ('\n', 'SPACE')]
```

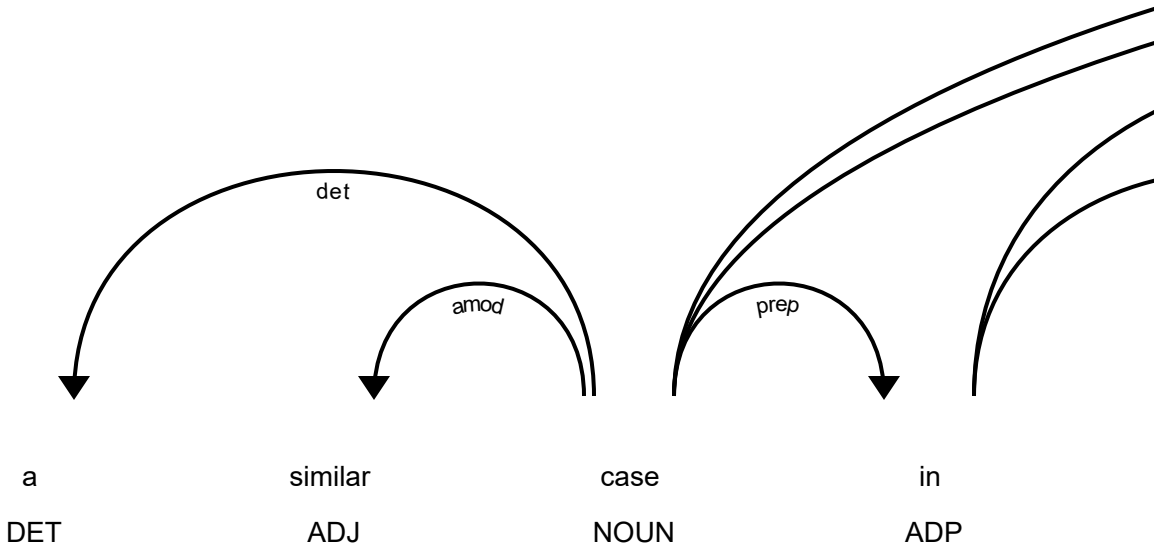



det
nummod
nmod
conj
cc
the
DET
six
NUM
phone
NOUN
and
CCONJ


nsubj
compound
nsubj
aux
Apple
state
it
have "
NOUN
NOUN
PRON
AUX


punct
prep
dep
pobj
"
in
August ,
PUNCT
SPACE
ADP
PROPN


relcl
nsubj
det
Samsung ,
PROPN
which
PRON
be
VERB
the
DET




```
[('the', 'det'), ('six', 'nummod'), ('phone', 'nmod'), ('and', 'cc'), ('tablet', 'co
nj'), ('affect', 'nsubj'), ('be', 'ROOT'), ('the', 'det'), ('Galaxy', 'compound'),
('S', 'compound'), ('III', 'attr'), (',', 'punct'), ('run', 'conj'), ('the', 'det'),
('new', 'amod'), ('Jelly', 'compound'), ('Bean', 'compound'), ('system', 'dobj'),
(',', 'punct'), ('the', 'det'), ('Galaxy', 'compound'), ('Tab', 'nmod'), ('8.9', 'nu
mmod'), ('Wifi', 'compound'), ('tablet', 'appos'), (',', 'punct'), ('the', 'det'),
('Galaxy', 'compound'), ('Tab', 'conj'), ('2', 'nummod'), ('10.1', 'appos'), (',',
'punct'), ('Galaxy', 'compound'), ('Rugby', 'compound'), ('Pro', 'appos'), ('and',
'cc'), ('Galaxy', 'compound'), ('S', 'compound'), ('III', 'compound'), ('mini', 'con
j'), ('.', 'punct'), ('\n', 'dep')]
```

small tip: You can use **sents = list(doc.sents)** to be able to use the index to access a sentence like **sents[2]** for the third sentence.

[total points: 7] Exercise 3: Comparison NLTK and spaCy

We will now compare the output of NLTK and spaCy, i.e., in what do they differ?

[points: 3] Exercise 3a: Part of speech tagging

Compare the output from NLTK and spaCy regarding part of speech tagging.

- To compare, you probably would like to compare sentence per sentence. Describe if the sentence splitting is different for NLTK than for spaCy. If not, where do they differ?

- Answer: NLTK splits on double colons `"`, while spaCy splits on full stops. Some prepositions are tagged differently, spaCy prefer to assign `IN` for all of them.

In [187…

```
for sent_nltk, sent_spacy in zip(sentences_nltk[1:], sentences_spacy[1:]):
    print(sent_nltk)
    print(sent_spacy)
    print()
```

```
The six phones and tablets affected are the Galaxy S III, running the new Jelly Bean
system, the Galaxy Tab 8.9 Wifi tablet, the Galaxy Tab 2 10.1, Galaxy Rugby Pro and
Galaxy S III mini.
The six phones and tablets affected are the Galaxy S III, running the new Jelly Bean
system, the Galaxy Tab 8.9 Wifi tablet, the Galaxy Tab 2 10.1, Galaxy Rugby Pro and
Galaxy S III mini.


Apple stated it had “acted quickly and diligently" in order to "determine that these
newly released products do infringe many of the same claims already asserted by Appl
e."
Apple stated it had “acted quickly and diligently" in order to "determine that these
newly released products do infringe many of the same claims already asserted by Appl
e.

In August, Samsung lost a US patent case to Apple and was ordered to pay its rival
$1.05bn (£0.66bn) in damages for copying features of the iPad and iPhone in its Gala
xy range of devices.
"
In August, Samsung lost a US patent case to Apple and was ordered to pay its rival
$1.05bn (£0.66bn) in damages for copying features of the iPad and iPhone in its Gala
xy range of devices.

Samsung, which is the world's top mobile phone maker, is appealing the ruling.
Samsung, which is the world's top mobile phone maker, is appealing the ruling.


A similar case in the UK found in Samsung's favour and ordered Apple to publish an a
pology making clear that the South Korean firm had not copied its iPad when designin
g its own devices.
A similar case in the UK found in Samsung's favour and ordered Apple to publish an a
pology making clear that the South Korean firm had not copied its iPad when designin
g its own devices.
```

In [188…

```
for pos_tags_per_sentence, sentences_spacy_lemmatized_tagged in zip(pos_tags_per_se
    print(pos_tags_per_sentence)
    print(sentences_spacy_lemmatized_tagged)
    print()
```

```
[('Apple', 'NNP'), ('stated', 'VBD'), ('it', 'PRP'), ('had', 'VBD'), ('“', 'NNP'),
('acted', 'VBD'), ('quickly', 'RB'), ('and', 'CC'), ('diligently', 'RB'), ("''",
"''"), ('in', 'IN'), ('order', 'NN'), ('to', 'TO'), ('``', '``'), ('determine', 'V
B'), ('that', 'IN'), ('these', 'DT'), ('newly', 'RB'), ('released', 'VBN'), ('produc
ts', 'NNS'), ('do', 'VBP'), ('infringe', 'VB'), ('many', 'JJ'), ('of', 'IN'), ('th
e', 'DT'), ('same', 'JJ'), ('claims', 'NNS'), ('already', 'RB'), ('asserted', 'VB
N'), ('by', 'IN'), ('Apple', 'NNP'), ('.', '.'), ("''", "'')]
[('Apple', 'NOUN'), ('state', 'NOUN'), ('it', 'PRON'), ('have', 'AUX'), ('"', 'PUNC
T'), ('act', 'VERB'), ('quickly', 'ADV'), ('and', 'CCONJ'), ('diligently', 'ADV'),
('"', 'PUNCT'), ('in', 'ADP'), ('order', 'NOUN'), ('to', 'PART'), ('"', 'PUNCT'),
('determine', 'VERB'), ('that', 'SCONJ'), ('these', 'DET'), ('newly', 'ADV'), ('rele
ase', 'NOUN'), ('product', 'NOUN'), ('do', 'AUX'), ('infringe', 'VERB'), ('many', 'A
DJ'), ('of', 'ADP'), ('the', 'DET'), ('same', 'ADJ'), ('claim', 'NOUN'), ('already',
'ADV'), ('assert', 'VERB'), ('by', 'ADP'), ('Apple', 'PROPN'), ('.', 'PUNCT')]

[('In', 'IN'), ('August', 'NNP'), (',', ','), ('Samsung', 'NNP'), ('lost', 'VBD'),
('a', 'DT'), ('US', 'NNP'), ('patent', 'NN'), ('case', 'NN'), ('to', 'TO'), ('Appl
e', 'NNP'), ('and', 'CC'), ('was', 'VBD'), ('ordered', 'VBN'), ('to', 'TO'), ('pay',
'VB'), ('its', 'PRP$'), ('rival', 'JJ'), ('$', '$'), ('1.05bn', 'CD'), ('(', '('),
('£0.66bn', 'NN'), (')', ')'), ('in', 'IN'), ('damages', 'NNS'), ('for', 'IN'), ('co
pying', 'VBG'), ('features', 'NNS'), ('of', 'IN'), ('the', 'DT'), ('iPad', 'NN'),
('and', 'CC'), ('iPhone', 'NN'), ('in', 'IN'), ('its', 'PRP$'), ('Galaxy', 'NNP'),
('range', 'NN'), ('of', 'IN'), ('devices', 'NNS'), ('.', '.')]
[('"', 'PUNCT'), ('\n ', 'SPACE'), ('in', 'ADP'), ('August', 'PROPN'), (',', 'PUNC
T'), ('Samsung', 'PROPN'), ('lose', 'VERB'), ('a', 'DET'), ('US', 'PROPN'), ('paten
t', 'NOUN'), ('case', 'NOUN'), ('to', 'ADP'), ('Apple', 'PROPN'), ('and', 'CCONJ'),
('be', 'AUX'), ('order', 'NOUN'), ('to', 'PART'), ('pay', 'VERB'), ('its', 'PRON'),
('rival', 'ADJ'), ('$', 'SYM'), ('1.05bn', 'NUM'), ('(', 'PUNCT'), ('£', 'NOUN'),
('0.66bn', 'NUM'), (')', 'PUNCT'), ('in', 'ADP'), ('damage', 'NOUN'), ('for', 'AD
P'), ('copy', 'NOUN'), ('feature', 'NOUN'), ('of', 'ADP'), ('the', 'DET'), ('iPad',
'PROPN'), ('and', 'CCONJ'), ('iPhone', 'PROPN'), ('in', 'ADP'), ('its', 'PRON'), ('G
alaxy', 'PROPN'), ('range', 'NOUN'), ('of', 'ADP'), ('device', 'NOUN'), ('.', 'PUNC
T')]

[('Samsung', 'NNP'), (',', ','), ('which', 'WDT'), ('is', 'VBZ'), ('the', 'DT'), ('w
orld', 'NN'), ("'s", 'POS'), ('top', 'JJ'), ('mobile', 'NN'), ('phone', 'NN'), ('mak
er', 'NN'), (',', ','), ('is', 'VBZ'), ('appealing', 'VBG'), ('the', 'DT'), ('rulin
g', 'NN'), ('.', '.')]
[('Samsung', 'PROPN'), (',', 'PUNCT'), ('which', 'PRON'), ('be', 'VERB'), ('the', 'D
ET'), ('world', 'NOUN'), ("'s", 'PART'), ('top', 'ADJ'), ('mobile', 'ADJ'), ('phon
e', 'NOUN'), ('maker', 'NOUN'), (',', 'PUNCT'), ('be', 'AUX'), ('appeal', 'VERB'),
('the', 'DET'), ('ruling', 'NOUN'), ('.', 'PUNCT'), ('\n', 'SPACE')]

[('A', 'DT'), ('similar', 'JJ'), ('case', 'NN'), ('in', 'IN'), ('the', 'DT'), ('UK',
'NNP'), ('found', 'VBD'), ('in', 'IN'), ('Samsung', 'NNP'), ("'s", 'POS'), ('favou
r', 'NN'), ('and', 'CC'), ('ordered', 'VBD'), ('Apple', 'NNP'), ('to', 'TO'), ('publ
ish', 'VB'), ('an', 'DT'), ('apology', 'NN'), ('making', 'VBG'), ('clear', 'JJ'),
('that', 'IN'), ('the', 'DT'), ('South', 'JJ'), ('Korean', 'JJ'), ('firm', 'NN'),
('had', 'VBD'), ('not', 'RB'), ('copied', 'VBN'), ('its', 'PRP$'), ('iPad', 'NN'),
('when', 'WRB'), ('designing', 'VBG'), ('its', 'PRP$'), ('own', 'JJ'), ('devices',
'NNS'), ('.', '.')]
[('a', 'DET'), ('similar', 'ADJ'), ('case', 'NOUN'), ('in', 'ADP'), ('the', 'DET'),
('UK', 'PROPN'), ('find', 'NOUN'), ('in', 'ADP'), ('Samsung', 'PROPN'), ("'s", 'PAR
T'), ('favour', 'NOUN'), ('and', 'CCONJ'), ('order', 'NOUN'), ('Apple', 'PROPN'),
('to', 'PART'), ('publish', 'VERB'), ('an', 'DET'), ('apology', 'NOUN'), ('making',
'NOUN'), ('clear', 'ADJ'), ('that', 'SCONJ'), ('the', 'DET'), ('south', 'ADJ'), ('ko
```

```
rean', 'ADJ'), ('firm', 'NOUN'), ('have', 'AUX'), ('not', 'PART'), ('copy', 'VERB'),
('its', 'PRON'), ('iPad', 'PROPN'), ('when', 'SCONJ'), ('design', 'VERB'), ('its',
'PRON'), ('own', 'ADJ'), ('device', 'NOUN'), ('.', 'PUNCT')]
```

- After checking the sentence splitting, select a sentence for which you expect interesting results and perhaps differences. Motivate your choice.

- Answer: The 3rd sentence is interesting because it contains a lot of named entities, including giant companies and technology inventions.

- Compare the output in `token.tag` from spaCy to the part of speech tagging from NLTK for each token in your selected sentence. Are there any differences? This is not a trick question; it is possible that there are no differences.

- Answer: The split are consistent, but the tags are different. For example, NLTK tags "Apple" as `NNP` while spaCy tags it as `PROPN`.

[points: 2] Exercise 3b: Named Entity Recognition (NER)

- Describe differences between the output from NLTK and spaCy for Named Entity Recognition. Which one do you think performs better?

- Answer: spaCy is better recognized named entities than NLTK. For example, NLTK does not recognize " South Korea" as a named entity, while spaCy does. spaCy seem to consider more context when recognizing named entities.

[points: 2] Exercise 3c: Constituency/dependency parsing

Choose one sentence from the text and run constituency parsing using NLTK and dependency parsing using spaCy.

- describe briefly the difference between constituency parsing and dependency parsing
- describe differences between the output from NLTK and spaCy.

- Answer: Constituency parsing is a tree representation of the whole context, while dependency parsing represents the relationships between words directly using dependencies. SpaCy provides a more comprehensive dependency parsing result than NLTK. For example, spaCy can recognize 7 relationships on word "lost" in the 3rd sentence, while NLTK only recognizes 2 relationships.

End of this notebook